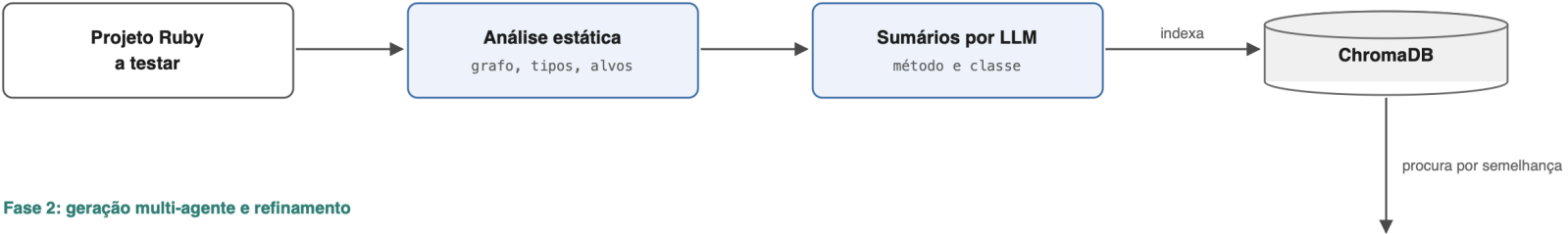


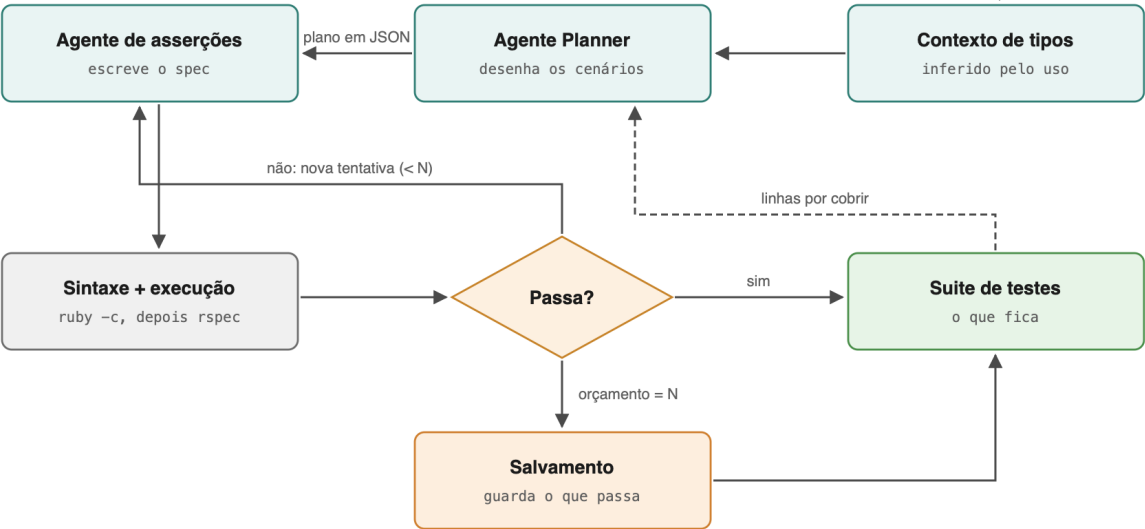
Arquitetura da ferramenta

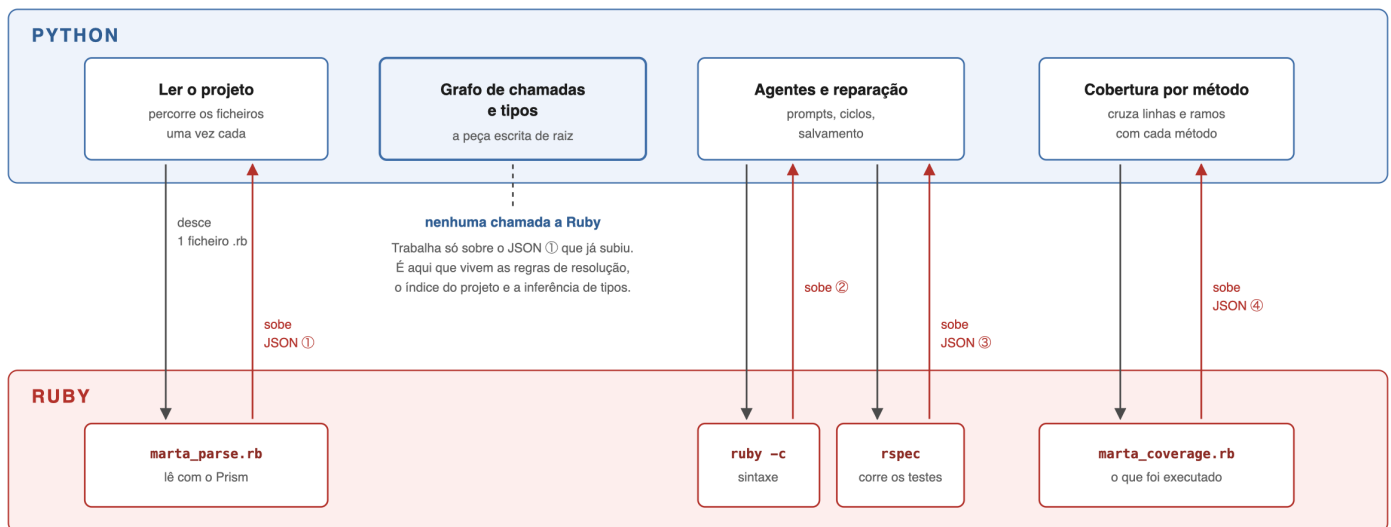
Fase 1: construção do contexto (offline, em cache)



Fase 2: geração multi-agente e refinamento

Validação e retenção





O Ruby nunca decide nada: só lê ficheiros, verifica sintaxe, corre testes e reporta o que executou.

O que sobe em cada uma dessas chamadas

① marta_parse.rb a estrutura do ficheiro

```
{
  "classes": [
    {
      "qualified_name": "Carteira",
      "superclass": null
    }
  ],
  "methods": [
    {
      "name": "saldo",
      "owner": "Carteira",
      "start_line": 5,
      "end_line": 7
    }
  ],
  "calls": [
    {
      "name": "total",
      "recv": "ivar",
      "recv_name": "@mov",
      "line": 6
    }
  ],
  "errors": []
}
```

O campo "recv" é o que diz sobre o quê a chamada foi feita. É dele que sai o grafo.

② ruby -c o único que não devolve JSON

Syntax OK

ou: x.rb:4: syntax error, unexpected end

③ rspec -f json estado e linha de cada teste

```
{
  "examples": [
    {
      "description": "positivo",
      "status": "passed",
      "line_number": 3
    }
  ],
  "summary": {
    "example_count": 1,
    "failure_count": 0
  }
}
```

Quando falha, traz também a mensagem do erro e a linha exata onde rebentou.

④ marta_coverage.rb o que correu, linha a linha e ramo a ramo

```
{
  "files": {
    "calc.rb": {
      "lines": [1, 1, 1, null, null],
      "branches": [[3, 1], [3, 0]]
    }
  }
}
```

lines: cada posição é uma linha do ficheiro; null = linha não executável.

branches: na linha 3 um lado do if correu 1 vez e o outro nunca. Por linhas parecia 100%.

Decisões:

Escolha do parser

O que é e para que serve: Um parser lê o texto de um ficheiro Ruby e devolve a sua estrutura. Preciso dele para quatro coisas:

- Estrutura dos ficheiros sem a sua execução

- Localização de métodos (diz-nos exatamente em eu linha um método começa e termina). Essencial para as re-runs guiadas por cobertura.
- Parâmetros e os métodos que ele chama. Para a inferência de tipo. Coisas como .toString associar o parâmetro uma string
- Saber que chamadas cada método faz e sobre que recetor foram feitas

Porquê o Prism. É o parser do núcleo da linguagem: parser por omissão do interpretador desde o Ruby 3.4. Só conhece a gramática do 3.3 e posterior, mas não faz mal proque código escrito para versões anteriores continua a ser lido pela gramática nova. O problema seria o contrário: um parser antigo não lê sintaxe recente.

Opção	Porque não
Prism	escolhido
Ripper	A árvore são vetores encaixados sem objetos nem tipos, não nos dá o que quero Só start, não end
parser	Anunciado que novas versões deixam de ser suportadas pela existência do prism
ruby_parser	Só tem suporte até ruby 3.0 e relata problemas em relação á localização dos métodos nos ficheiros
RubyVM: :AbstractSyntaxTree	A documentação diz que é experimental e sem garantia de compatibilidade. Nodes sao indexados não dá para relacionar pelos nomes

Como ele trabalha

Respondo a analisar a imagem 2

Escolha da framework dos testes

O que é e para que serve. É a framework em que os testes gerados são escritos e executados. Preciso dela para quatro coisas:

- Correr um ficheiro de testes isolado, sem depender da configuração de cada projeto
- Saber, por teste, se passou ou falhou
- A mensagem do erro, que segue no contexto da ronda seguinte para o modelo reparar o teste
- A linha em que o teste falhou, para o salvamento poder descartar só o it que falhou e manter os restantes

Porquê o RSpec. Dá tudo isto. A opção -f json devolve:

status : "failed"

line_number : 3 (o bloco it)

exception : "expected: 99 got: 5"

backtrace[0] : ".../f_spec.rb:4:in ..." (a linha + que rebentou)

Opção	Porque não
RSpec	É a escolhida
Minitest (Ruby integrated)	Não reporta estruturado como quero. Embora embutido, exige um plugin externo
test-unit	Menos comum das 3 e com o problema de saída do minitest

grafo de chamadas

O que é e para que serve. Responde a *quem chama quem*. Ao gerar testes para um método, junto ao contexto os métodos que ele usa por dentro.

O exemplo que vamos seguir do princípio ao fim

```
class Carteira
  def initialize
    @mov = Movimento.new
  end
  def saldo
    @mov.total
  end
end
```

A laranja: aqui fica-se a saber o que @mov é.

A vermelho: a chamada que queremos resolver.

1 O Prism lê o ficheiro

devolve a estrutura: classes, métodos, e todas as chamadas

2 Em cada chamada, olha para o RECETOR

o nome invocado é **total** , e foi invocado sobre **@mov**

O nome sozinho não chega: pode haver dezenas de métodos "total" no projeto.

3 o recetor só pode ter quatro formas, e é a forma que diz quanto sabemos sobre ele

FORMA DO RECETOR	APARECE COMO	DE ONDE VEM A INFORMAÇÃO	CERTEZA
o próprio objeto	total	estamos dentro da classe Carteira, logo é a Carteira	certa
uma constante	Movimento.total	o nome escrito no código É a classe. Não há nada a deduzir	certa
variável de instância	@mov.total	o construtor guardou lá um Movimento, e nós registámos isso	alta ← o nosso caso
parâmetro ou variável local	x.total	ninguém diz o que x é. Procuramos a classe que responde a tudo o que se lhe faz	baixa

É sempre a mesma chamada, ao método **total** . O que muda é só sobre o quê, e é isso que decide o que conseguimos saber.



O grafo completo é isto repetido para todas as chamadas de todos os ficheiros: na biblioteca money são 231 arestas entre 149 métodos.

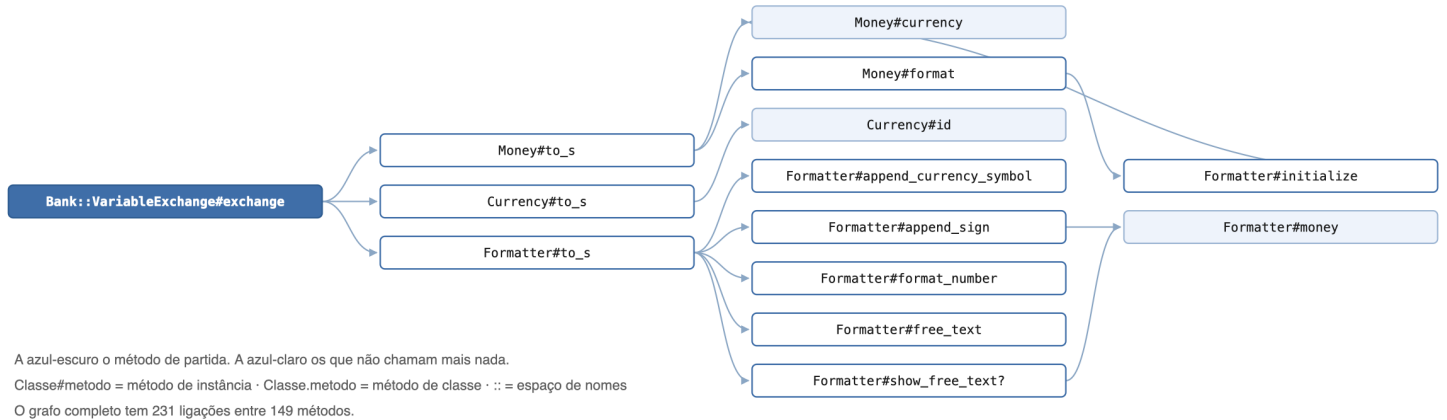
Porquê construí o meu. Existem ferramentas de grafo de chamadas para Ruby. Nenhuma cumpre o que preciso.

Ferramentas de grafo de chamadas para Ruby

Abordagem	Exemplos	O que falha
Dinâmicas	rubydeps, code_mapper, visual-call-graph	Precisam de correr o código, e só veem o que for executado. O rubydeps corre a suite de testes do projeto e fica ao nível da classe
Estáticas,	code-explorer,	Não chegam ao método em projeto inteiro: o

âmbito limitado	rubrowser	code-explorer faz o grafo dentro de um ficheiro, o rubrowser liga classes e módulos
A minha	—	Ao nível do método, pré-execução e para o projeto inteiro

Excerto real do grafo da biblioteca money: 14 dos 149 métodos



Inferência de tipos

É o projeto todo de uma vez, antes de se gerar teste nenhum, na análise estática, sem custar nenhuma chamada ao modelo.

Em Python o programador pode declarar o tipo na própria assinatura (def somar(a: int, b: int)). Em Ruby existem anotações mas praticamente ninguém as faz

RBS, oficial / biblioteca padrão desde o Ruby 3.0, mas as assinaturas vivem em ficheiros .rbs à parte e o Sorbet, que precisa de uma gem e de assinaturas escritas à mão.

Resta inferir pelo comportamento, em dois tempos:

1. Ao ler cada ficheiro no parser inicial, registo para cada parâmetro os métodos que lhe são chamados no corpo e vou construindo o índice das classes do projeto e dos métodos a que cada uma responde, contando os herdados.

2. No fim, com o índice completo, procuro as classes que respondem a todos os métodos exigidos por aquele parâmetro. Essas são as candidatas.

INFERRED PARAMETER TYPES:

- `moeda` responds to [iso_code, to_s] -> likely: Money::Currency

O cruzamento só pode ser feito depois de todos os ficheiros lidos, porque a classe candidata está quase sempre noutra ficheiro. Quando nenhuma corresponde, entrego ao modelo só a lista de métodos exigidos sem palpite de tipo.

Os prompts

Planner

Analyze the following Ruby method and generate a comprehensive test plan.

TASK: Generate 3 distinct test scenarios covering:

CRITICAL: If 'MISSING LINES TO COVER' are provided in the context, you MUST design these scenarios specifically to execute those missing lines.

1. Valid inputs (Happy Path)
2. Edge cases (e.g., nil, empty arrays, boundary values)
3. Invalid inputs / error handling (e.g. raising an error)

SETUP GUIDANCE: prefer REAL objects with simple concrete values in 'setup'; only suggest doubles/stubs for true external I/O (network, filesystem, subprocess).

OUTPUT FORMAT: Return ONLY a raw JSON list. No Markdown. No Explanations.

Agente de asserções

RULES:

1. Output ONLY Ruby code in a ```ruby``` block (a complete, self-contained spec file).
2. Start the file with: require "<target>"
3. Use RSpec: RSpec.describe <Subject> do ... end with one independent it "..." do ... end block per scenario.
4. Use expect(...).to eq(...) / raise_error(...) matchers.
5. AVOID MOCKS unless strictly necessary: prefer constructing REAL objects with simple values. NEVER mock the class under test, plain value objects, or anything you can instantiate directly. A wrong double fails the test without testing anything.
6. IF you must isolate a dependency, use RSpec doubles with allow(...).to receive(...). NEVER redefine global constants or monkey-patch classes directly.
7. EXPECTATIONS: keep each it block focused, 1 to 2 expect calls per block, asserting CONCRETE expected values you derived from the source code.

Na reparação, o prompt do segundo agente passa a incluir a mensagem de erro da tentativa anterior e exemplos de métodos parecidos com esse erro, procurados no ChromaDB.

Escolha para medição de cobertura

O que é e para que serve. Conta quantas vezes cada linha foi executada e quantas vezes cada ramo de uma decisão foi tomado. É daqui que saem as linhas em falta de cada método, que alimentam a ronda seguinte.

Porquê o Coverage embutido. Dá-nos exatamente isso, sem dependência nenhuma. As alternativas acrescentam coisas de que não preciso.

Opção	Porque não
Coverage	É o escolhido: sem dependências e dá acesso direto aos dados em bruto
SimpleCov	Pega nos dados do Coverage e gera html com os ficheiros e as linhas coloridas
DeepCover	Mais preciso, mede dentro de cada linha. A documentação diz que não está preparado para ruby 3.x e pode dar problemas

Benchmark: como o dataset foi construído



Cada camada tem um critério objetivo, aplicado por script. As três primeiras eliminam, a quarta ordena, a quinta confirma. Todo o processo é re-executável: clona, mede e apaga, sem instalar nada e sem executar o código dos projetos.

Camada 1: a população de partida

O que . Descarreguei o awesome-ruby, extraí todas as entradas que apontam para repositórios do GitHub, e consultei o RubyGems para saber quantas descargas tem cada uma. Fiquei com as que estão acima de 100 milhões de descargas.

Resultado: 840 candidatas na lista, 125 gems acima do limiar, distribuídas por 55 categorias.

RubyGems é o repositório central de bibliotecas de Ruby, o equivalente ao PyPI do Python. Publico ao número de descargas de cada gem

awesome-ruby é uma lista curada pela comunidade, mantida no GitHub, que organiza as gems por categoria (clientes HTTP e por aí fora).

Excluí também secções inteiras do awesome-ruby. Ou não têm código (livros, podcasts,etc), ou não são bibliotecas Ruby.

Se a gem e o repo tiverem nomes diferentes o cruzamento falha.

Camada 2: diagnosticar cada gem, uma a uma

Como? Por cada uma das 125: clonar, analisar estaticamente com o mesmo parser que a ferramenta usa. Não valia. Apenas executar cada um deles que ia ter de estar a instalar dependências, etc.

Critérios de inclusão:

Critério	Como é verificado	Porquê
Tem dimensão mínima	pelo menos 15 métodos-alvo	gems pequenas de mais não servem
O parser consegue ler	zero erros do Prism	se não consigo ler o código, não posso gerar testes para ele

Das 125 são analisadas e 120 passam os três critérios em conjunto.

As 5 exclusões:

6 a contar com a camada 3 que não faz sentido ser a parte

gem	ficheiros	métodos	erros	motivo
kaminari	2	0	0	menos de 15 métodos
sentry-ruby	1	2	0	menos de 15 métodos
rqrcode	9	6	0	menos de 15 métodos
jaro_winkler	4	7	0	menos de 15 métodos
docile	6	9	0	menos de 15 métodos

Camada 3: filtrar o que não serve para medir

Gems gigantes com Código repetido

twilio-ruby tem 22 047 métodos, mas não é um projeto complex. Métodos são praticamente todos iguais. Uma gem destas dominaria o corpus sem qualquer sentido para tal.

Meti teto de 2000 métodos!

1 gem excluída. Restam 94 elegíveis.

Camada 4: Filtrar por diversidade

7 características medidas

Cada uma corresponde a uma pode dar target a um escopo diferente da ferramenta

Característica	Como é calculada	Que parte da ferramenta exercita
Número de métodos	contagem dos métodos	escala e custo
Linhas por método	linha final menos linha inicial, em média	o contexto que vai no prompt
% de métodos singleton	métodos de classe a dividir pelo total	regime de programação: funções de módulo contra objetos com estado
% de métodos com duck typing	métodos cujos parâmetros têm chamadas registadas sobre eles	a inferência de tipos
Mixins por classe	include + extend + prepend a dividir pelo número de classes	o grafo de chamadas e a ordem de procura de métodos
Profundidade de herança	comprimento máximo da cadeia de ancestrais	a resolução de chamadas em hierarquias
Metaprogramação por 100 métodos	contagem de chamadas numa lista fixa de 22 nomes	o limite da análise estática

As sete características são normalizadas (escala logarítmica para tamanho e profundidade).

Eu escolhi ter 12 projetos

farthest-first traversal

Cada gem passa a ser um ponto num espaço de 7 dimensões, a primeira escolhida é a mais longe do centro desse espaço e as seguintes são as mais longe da gem escolhida anteriormente até chegar ao teto

Teto de 2 gems por categoria, porque se não uma categoria podia dominar o corpus

"um conjunto de 12 que cobre o espaço de diversidade" e nunca "as 12 mais diversas".

Camada 5: confirmar instalações

O que foi feito: Percorrer candidatas: clonar, registar o commit exato, instalar as dependências de execução.

Ficam as 12 primeiras que instalam limpo.

Nesta passagem caíram duas gems, ambas por extensões nativas: a mysql2 e a bootsnap

O corpus final

gem	categoria	métodos	linhas/mét.	singleton %	duck %	metaprogramming/100	commit
rdoc	documentation	1779	18.3	5	35	1.3	c79013652d
faker	testing	1381	4.8	98	11	0.9	cca4184947
rouge	code highlighting	646	7.9	60	26	2.8	bd7dad7b09
gitlab	third-party apis	644	4.0	4	11	1.4	8aef58f39f
sprockets	assets	485	8.4	11	32	1.9	16921a5491
kramdown	markdown processors	328	10.8	10	62	5.8	12b3d2e752
virtus	core	204	4.9	30	40	28.4	fce56bd667

	extensions						
addressable	core extensions	132	18.2	23	52	2.3	d298c9f551
timecop	testing	69	6.3	49	13	1.4	b3c9a5a6f0
parallel	concurrency and parallelism	53	11.5	75	53	9.4	9bc03fe4e8
colorize	cli utilities	36	5.7	3	19	8.3	a252d8af6b
formatador	cli utilities	20	10.8	0	50	0.0	b059e42ee5

Amplitude conseguida: métodos de 20 a 1779 · linhas por método de 4,0 a 18,3 · singleton de 0% a 98% · duck typing de 11% a 62% · metaprogramação de 0,0 a 28,4 por 100 métodos · 9 categorias distintas · 5777 métodos no total.

Os dois extremos que valem a pena apontar numa apresentação: a formatador é código direto, sem metaprogramação nenhuma. A virtus constrói os seus métodos em tempo de execução, com 28,4 chamadas de metaprogramação por cada 100 métodos. É exatamente essa distância que quero.

Camada 6: escolher os ficheiros-alvo dentro de cada projeto

Testar todos os métodos dos 12 projetos são 5777 alvos, cerca de 114 horas de LLM por execução. É impraticável, e desnecessário.

O estado da arte faz o mesmo. O benchmark Python que uso, herdado do CodaMosa, tem 486 módulos em 27 projetos, e não os projetos inteiros. Nem todo o ficheiro vale a pena testar.

Critérios de inclusão de um ficheiro:

Critério	Valor	Porquê
Mínimo de métodos	3	ficheiros triviais não medem nada
Máximo de métodos	120	o teto é alto de propósito: o sinal de ficheiro gerado não é o tamanho, é a baixa densidade de lógica
Densidade de lógica mínima	35% dos métodos	filtra ficheiros de dados e de constantes, que não exercitam a ferramenta
Caminhos excluídos	version.rb, locale/, data/, generated/, vendor/, templates/	quase nunca contêm lógica testável

Como se mede 'ter lógica'. Um método conta como tendo lógica se ocupa pelo menos duas linhas e se faz duas ou mais chamadas, ou pelo menos uma chamada de uma lista de nomes que denunciam ramificação: raise, map, select, reject, each, reduce, detect, find, sort, gsub, match, split, fetch.

Os ficheiros elegíveis são depois ordenados por densidade de lógica, e escolhem-se os primeiros até esgotar o orçamento de métodos do projeto. O primeiro ficheiro entra sempre, mesmo que

sozinho exceda o orçamento — caso contrário gems de ficheiro único ficariam sem alvo nenhum.

Um erro que valeu a pena corrigir: o teto de métodos por ficheiro estava em 40 e excluía a parallel, que é uma gem de ficheiro único com 51 métodos, 44 deles com lógica. Foi isso que nos levou a perceber que o critério certo é a densidade de lógica, e não o tamanho.